Table des matières

Data Engineering — plan des modules 1
1. Ce que cette formation est, et ce qu’elle n’est pas 1
2. Ce qui entre, ce qui sort 2
3. La pile et le poste de travail 4
4. Le projet de l’apprenant 7
5. Les recoupements 7
6. Les douze cours 9
7. Le capstone 21
8. La synthèse, et le mini-cas 22
9. Le kit 02 25

Data Engineering — plan des modules

Écrit le 2026-08-31, amendé le 2026-09-11 pour les données industrielles, révisé le 2026-09-15, tenu à jour jusqu’à la fin de la rédaction, le 2026-09-23. Ce fichier n’est pas un support de cours : c’est le plan de la formation, ce qu’elle promet, ce qu’elle livre et dans quel ordre. Il se lit pour juger la conception plutôt que pour apprendre.

Ce que la formation est. Douze cours, chacun en trois supports, sur un seul cas d’étude, Véhicules Boréal inc. La formation livre des produits nommés (la couche `RAW`, les séries de l’usine) que les formations suivantes consomment. Chaque cours dit ses données, ses événements plantés et ses variantes Snowflake et Databricks. Un capstone, le mini-cas de La Brûlerie, la synthèse Aurore et le kit 02 ferment la formation.

Les chemins sont relatifs à la racine du parcours. Les numéros D désignent une décision de conception, citée pour que chaque choix reste traçable ; les numéros E, un événement planté de `_univers/boreal/README.md`. Les chiffres cités sont la sortie réelle du générateur, lue dans `_univers/boreal/CHIFFRES_ATTENDUS.md`.

---

1. Ce que cette formation est, et ce qu’elle n’est pas

Elle couvre l’amont de la chaîne : faire arriver la donnée, la ranger, la faire tourner à l’heure, et prouver qu’elle est bonne. Elle s’arrête là où commence la transformation métier : à la couche `RAW` comprise pour les sources relationnelles et les événements, aux séries nettoyées pour les données de terrain (D32).

© 2026 iAlexMG. Tous droits réservés.

Formation	Ce qu'elle possède	Frontière avec Data Engineering
SQL avancé	le langage	DE écrit du SQL sans l'enseigner ; chaque notion SQL reçoit un renvoi nommé
Snowflake	la plateforme d'entrepôt	DE y livre dans ses annexes, sans l'administrer
Analytics Engineering	la transformation et la modélisation	DE livre la `RAW` et les séries de l'usine ; AE commence au `STAGING`. DE ne décode pas les codes SAP, ne dédoublonne pas les clients et ne joint pas deux sources
Business Intelligence	la restitution	DE monte Grafana sur la base de séries temporelles et y branche l'alerte ; la lecture du tableau de bord appartient à BI
Data Science	la modélisation statistique	DE sert les séries longues de l'usine et la météo, sans construire de modèle
Machine Learning Engineering	le pipeline de modèle	MLE réemploie les conteneurs et Airflow de DE ; DE ne sert aucun modèle
AI Engineering	la récupération augmentée	DE dépose le corpus tel quel dans le lac ; le découpage et l'indexation appartiennent à AI
Architecture et Gouvernance	les règles et la cible	DE écrit et exécute les contrats techniques ; Archi en fait des engagements entre équipes
Linux, Python	le système, le langage	DE suppose le terminal et la syntaxe acquis

La règle des renvois de `_commun/STYLE.md`, section 5, s'applique : une notion qui appartient à une autre formation n'est pas réexpliquée, elle est renvoyée nommément.

2. Ce qui entre, ce qui sort

2.1 Les entrées

Les six sources de Boréal, telles que ses systèmes les livrent, fabriquées par `_univers/boreal/generer_boreal.py` dans `Projet_Boreal/sources/`. DE ne consomme le produit d'aucune autre formation : le générateur lui suffit, et aucune porte d'entrée n'est à écrire (D12).

© 2026 iAlexMG. Tous droits réservés.

Source	Ce qui arrive dans `sources/`	Défauts qui servent DE	Cours
ERP	`erp/extraits/AAAA-MM/`, un CSV par table SAP ; `erp/postgresql/`, la copie de la base et `charger_erp.sql`	l'extrait de mars 2023 livré deux fois (E6), une commande avant la fiche de son concessionnaire (E7), des textes `NUMC` à zéros de tête et des dates `DATS` à `00000000`, les documents de modification (E8), les mois de 2026 (E17)	01, 03, 04, 06, 11
Usine	`usine/historien/`, un CSV par jour ; `usine/referentiel/` ; `usine/gmao/` ; `usine/compteur/`, un CSV par mois	valeurs de départ, archivage forcé de minuit, heure locale sans fuseau et collisions (E1), capteur figé (E2), coupures et valeurs aberrantes (E3), dérives et pannes (E4, E5), dates `JJ/MM/AAAA HH:MM`, compteur horodaté en fin d'intervalle avec son décalage à UTC, relevés estimés	01, 02, 07, 08, 09, 11
Garanties	`garanties/AAAA-MM/` : clients, enregistrements, visites et consentements en CSV, réclamations en JSON	données personnelles, une réclamation qui revient à chaque changement de statut, un effacement (E14)	03, 11
Télémétrie	`telemetrie/sorties_AAAA-MM-JJ.jsonl`, ce qu'écrit l'abonné du courtier MQTT	doublons de la qualité de service 1, messages en retard (E12), un champ ajouté et un champ renommé (E11)	01, 02, 03, 09, 10, 11
Corpus	`corpus/` : documents HTML et Markdown, catalogue `index.csv`	des documents qui paraissent à leur date	03
Météo	`meteo/`, l'instantané figé d'ECCC ; la vraie API au Cours 04	trous et drapeaux de qualité de la vraie donnée	03, 04

2.2 Les sorties

© 2026 iAlexMG. Tous droits réservés.

Produit	Forme	Où	Consommé par	Kit
La couche `RAW`	une table par fichier livré, en texte, avec ses colonnes de lignage (D60)	schéma `raw` de l'entrepôt DuckDB, et une copie Parquet par table	AE, cours du `STAGING` ; Archi	02
Les séries de l'usine	les étiquettes de l'historien et l'énergie du compteur, nettoyées et rééchantillonnées	`plateforme/lac/argent/`, et schéma `argent` de l'entrepôt (D61)	AE (étoile de l'usine) ; DS (maintenance prédictive, prévision de la demande) ; MLE	02, et 04 avec l'historien complet
Le lac bronze et argent	Parquet partitionné, puis tables Iceberg	`plateforme/lac/`	Archi, Cours 01	07
La base de séries temporelles et le flux rejouable	TimescaleDB chargée, le rejoueur MQTT de `_univers/rejoueurs/`	le `docker compose` du `Solution/` de DE	BI, son cours Grafana	03
Le corpus déposé tel quel	les fichiers, et `raw.corpus_index`	`plateforme/lac/bronze/corpus/`	AI, Cours 02	06
Les contrats de données	YAML et requêtes de contrôle	le dépôt `01_de`	AE (ses tests prennent le relais) ; Archi, Cours 05	07
Le DAG et les conteneurs	DAG Airflow, `Dockerfile`, `docker compose`	`02_Boreal/Solution/`	MLE, Cours 01 et 04	aucun : de l'outillage

Chaque produit est décrit dans `02_Boreal/Solution/LIVRAISON.md` : ses tables, ses colonnes de lignage, ses comptes attendus, le cours qui le fabrique et la commande du kit qui le reconstruit. C'est le document que les formations suivantes citent quand elles nomment ce qu'elles reçoivent (D12). Il commence au Cours 03 et se complète au Cours 12.

3. La pile et le poste de travail

3.1 La pile

Python et des outils ouverts, en local (D13). DuckDB est le moteur de référence des labos : il lit CSV, JSON, Parquet et Iceberg sans serveur. Les services tournent en conteneurs. Spark et Kafka ne sortent que dans les cours dont ils sont le sujet, et rien n'oblige à ouvrir un compte infonuagique.

© 2026 iAlexMG. Tous droits réservés.

Outil	Rôle	Où il tourne	Dès le cours
Python 3.11, environnement conda `formations`	le code du pipeline	Windows	01
DuckDB, paquet et ligne de commande	le SQL des labos, l'entrepôt local	Windows	01
Git	le dépôt `01_de`	Windows	01
Docker Engine, `docker compose`	les services	l'Ubuntu de WSL 2 (D59)	04
PostgreSQL 18	la base de l'ERP de Boréal	conteneur, port 5433 : le 5432 reste à un PostgreSQL déjà installé, celui de la formation SQL avancé par exemple	04
psqlODBC, `pyodbc`	l'extraction par ODBC	Windows	04
Airflow 3	l'orchestration	conteneurs	06
Spark 4	le traitement distribué	conteneur : sous Windows, Spark demande une version de Java précise et des utilitaires Hadoop (à revérifier à l'essai)	07
TimescaleDB	la base de séries temporelles	conteneur	08
Mosquitto, Kafka en mode KRaft	le chemin chaud	conteneurs	09
PyIceberg, `deltalake`	les formats de table ouverts	Windows	10
Grafana	le tableau de bord du chemin chaud	conteneur	12

3.2 Le poste de l'auteur, constaté le 2026-09-15

Élément	État
Windows	10 Famille, 22H2 (build 19045)
Mémoire	32 Go, dont 15 Go laissés à WSL 2
WSL 2	version 2.7, Ubuntu 26.04 LTS, systemd actif, sans Docker

© 2026 iAlexMG. Tous droits réservés.

Élément	État
Docker	absent ; des restes d'un Docker Desktop désinstallé (la distribution `docker-desktop` de WSL et deux dossiers de configuration)
PostgreSQL	18, installé en natif
DuckDB	1.5.5, paquet et ligne de commande
Java	23

Faits vérifiés le 2026-09-15 : Airflow ne tourne pas nativement sous Windows (WSL 2 ou conteneurs, selon sa documentation) ; Kafka non plus, en pratique ; Docker ne soutient Docker Desktop que sur les versions de Windows encore dans le calendrier de service de Microsoft, que Windows 10 a quitté le 2025-10-14, et sa page d'installation ne nomme que les éditions Pro, Entreprise et Éducation.

3.3 Les variantes Snowflake et Databricks

La règle est celle de tout le parcours. Une annexe quand le lecteur fait autre chose, un encadré quand la plateforme change un concept sans changer le geste. Les faits de chaque plateforme se revérifient avant d'écrire, aucun script ne porte d'identifiant de compte, et c'est vous qui les exécutez.

Cours	Snowflake	Databricks	Forme
02 Formats de fichiers et lac bronze	micro-partitions	fichiers Delta	encadré
03 Charger des fichiers dans la `RAW`	stage interne, `COPY INTO`, Snowpipe, schéma transient ; les quatre remarques de relecture d'AE sur le stage, le schéma transient et les rôles	volume Unity Catalog, `COPY INTO`, Auto Loader	annexe
06 Orchestration	tâches	Jobs	encadré
07 Spark	Snowpark	Spark serverless de Free Edition, le terrain même du cours	annexe Databricks, encadré Snowflake
09 Streaming	Snowpipe Streaming, Streams et Tasks	Structured Streaming	encadré
10 Formats de table ouverts	tables Iceberg	Delta, Unity Catalog ; encadré Fabric	annexe
11 Qualité	fonctions de mesure de la qualité	attentes des pipelines déclaratifs	encadré

© 2026 iAlexMG. Tous droits réservés.

Cours	Snowflake	Databricks	Forme
12 Capstone	clonage sans copie, graphe de tâches	tâches Lakeflow, tableaux de bord AI/BI, Declarative Automation Bundles	encadré

4. Le projet de l'apprenant

Décision D33 : un seul dossier `Projet_Boreal/`, ignoré par le dépôt des formations, qui porte une plateforme partagée et un dépôt Git par formation.

Dossier	Contenu	Qui l'écrit
`Projet_Boreal/sources/`	la sortie du générateur	`generer_boreal.py`
`Projet_Boreal/plateforme/lac/bronze/`, `argent/`	Parquet, puis Iceberg	DE
`Projet_Boreal/plateforme/entrepot/`	l'entrepôt DuckDB en un seul fichier, `boreal.duckdb` (D67), schémas `raw` et `argent` ; les autres viendront avec AE et MLE	DE
`Projet_Boreal/01_de/`	votre dépôt : le code du pipeline, les DAG, le `docker-compose.yml`, les contrats, les tests	vous

La ligne de commande DuckDB se lance depuis `Projet_Boreal/`, et le code depuis `01_de/` (D62). Le mini-cas et la synthèse suivent la même forme dans `Projet_Brulerie/` et `Projet_Aurore/`, eux aussi ignorés par Git.

5. Les recoupements

Décision prise avec vous le 2026-08-31 : chaque théorie se termine par un encadré court, une ligne par formation voisine, qui dit ce qui est commun et qui possède la matière. Le contenu de chaque encadré est fixé ici.

Cours	Recoupe	Ce que l'encadré doit dire
01	AE, DS	Les métiers se partagent la même chaîne. DE livre la `RAW` et les séries de l'usine ; AE prend la suite au `STAGING`, DS consomme les séries.

© 2026 iAlexMG. Tous droits réservés.

Cours	Recoupe	Ce que l'encadré doit dire
02	Snowflake	Les micro-partitions de Snowflake sont le partitionnement d'un lac, appliqué en interne.
03	AE, Snowflake	`COPY INTO` et Snowpipe se voient côté plateforme dans la formation Snowflake ; ici, la mécanique générale. Les codes SAP que la `RAW` garde tels quels se décodent au `STAGING` d'AE.
04	SQL	Les Cours 10 et 11 de SQL possèdent le plan de requête et l'index, la lecture longue et les versions de lignes ; ici, ce qu'ils changent pour qui extrait d'une base.
05	Python, Linux	Le langage et le terminal appartiennent aux deux autres formations ; ici, seulement ce qui rend un pipeline rejouable.
06	AE, MLE	`dbt build` dans un DAG est le point de couture avec AE. Le DAG d'entraînement de MLE réemploie l'outil.
07	SQL	Le plan d'exécution de Spark et celui d'un moteur SQL résolvent le même problème. Renvoi au Cours 10 de SQL.
08	SQL, DS	Le Cours 09 de SQL possède les séries temporelles et les jointures as-of ; ici, d'où viennent les points et ce qu'ils valent. DS s'en sert pour la maintenance prédictive et la prévision de la demande électrique.
09	BI, Archi	Le chemin chaud finit dans Grafana, dont la lecture appartient à BI. Lambda et Kappa reviennent au Cours 01 d'Archi, comme choix d'architecture.
10	AE, Archi	Bronze, argent et or recoupent `RAW`, `STAGING` et `MARTS` ; le parcours donne l'argent des séries de terrain à DE (D32). Archi reprend le débat du lac et

© 2026 iAlexMG. Tous droits réservés.

Cours	Recoupe	Ce que l'encadré doit dire
		de l'entrepôt (Cours 01) et le choix de la pile (Cours 03).
11	AE, Archi	Les tests dbt sont la version aval des contrats posés ici. Archi fait du contrat un engagement, avec un propriétaire et un recours.
12	AE, BI, DS	La `RAW` du capstone est ce que le kit 02 livre à AE ; le tableau de bord Grafana passe à BI ; les séries de l'usine nourrissent DS.

6. Les douze cours

La formation compte douze cours numérotés 01 à 12, précédés d'un Cours 00 qui génère les données sources (2026-10-18) ; l'installation, elle, reste hors de la formation (D97). Les anciens Cours 01 et 02 se sont coupés chacun en deux le 2026-09-18, à la charnière que leurs fiches rappellent, et tout ce qui suivait a reculé de deux rangs. Un cours se numérote par sa place dans la formation, jamais par son histoire.

Avant la coupe	Numéro	Titre	Dossier	Coupure nommée
nouveau, 2026-10-18	00	Générer les données sources	`Cours_00_Generer_les_donnees_sources`	
01, première moitié	01	Analyser les sources	`Cours_01_Analyser_les_sources`	
01, seconde moitié	02	Ranger les fichiers en bronze	`Cours_02_Ranger_les_fichiers_en_bronze`	
02, première moitié	03	Charger la `RAW`	`Cours_03_Charger_la_RAW`	
02, seconde moitié	04	Extraire d'une base ou d'une API	`Cours_04_Extraire_d_une_base_ou_d_une_API`	
03	05	Rendre le pipeline robuste	`Cours_05_Rendre_le_pipeline_robuste`	
04	06	Orchestrer et rattraper	`Cours_06_Orchestrer_et_rattraper`	après l'étape 5

© 2026 iAlexMG. Tous droits réservés.

Avant la coupe	Numéro	Titre	Dossier	Coupure nommée
05	07	Traiter un gros volume	`Cours_07_Traiter_un_gros_volume`	
06	08	Lire des séries temporelles	`Cours_08_Lire_des_series_temporelles`	le terrain et ses pièges, puis l'argent
07	09	Ingérer un flux d'événements	`Cours_09_Ingerer_un_flux_d_evenements`	MQTT, puis Kafka
08	10	Passer à un format de table ouvert	`Cours_10_Passer_a_un_format_de_table_ouvert`	
09	11	Contrôler la qualité	`Cours_11_Controler_la_qualite`	
10	12	Rebâtir le pipeline de bout en bout	`Cours_12_Rebatir_le_pipeline_de_bout_en_bout`	

Chaque cours porte, depuis le 2026-10-18, le nom de son rang, le même dans les trois projets de la formation : un verbe, puis ce qu'on fait ; l'outil se dit dans le texte, pas dans le nom. Les quatre premiers disent encore la charnière de leur coupe : analyser puis ranger, fichiers puis base et API. Les cours qui n'ont pas de coupure nommée ici annoncent leurs séances quand ils s'écrivent ou se relisent, mesurées sur la frappe (D93). Le sous-titre des supports dit `Cours N/12`, et `Cours de départ` pour le Cours 00.

Le Cours 00 n'a qu'un support, `Cours_00_DE_Boreal.md`. Pour chacun des autres : trois supports (`Theorie_NN`, `Labo_NN`, `Exercices_corriges_NN`), cinq familles d'exercices et environ 40 exercices, comme chez AE ; un `Labo_NN.sql` généré ; aucune image (D21). Chaque labo se termine par un tableau d'état du pipeline (tables, lignes, contrôles), le même que le kit 02 affiche. Chaque cours ajoute sa part au `Solution/` de la formation, et ses chiffres sortent de l'exécution de ce `Solution/` sur une génération fraîche.

L'installation, hors de la formation

L'installation n'est plus un Cours 00 depuis le 2026-09-18 (D97). Le socle du poste vit dans `Data_Domain/00_Installation/Guide_installation.md`, commun au parcours : l'environnement conda et les paquets, Git, la ligne de commande DuckDB, Docker Engine dans l'Ubuntu de WSL 2 et le pilote psqlODBC. Ce que la formation allume elle-même ouvre le cours qui s'en sert, dans une section `## Avant de commencer`. Le projet et ses sources ont leur cours, le Cours 00, revenu le 2026-10-18 : ils se refont à chaque projet, c'est du travail et non de l'installation.

© 2026 iAlexMG. Tous droits réservés.

Ce qui s'allume	Où	Ce que ça monte
le projet et ses sources	Cours 00	`Projet_Boreal/` créé, puis la génération de Boréal : 1 763 fichiers, 138 Mo, un peu plus d'une minute, verdict « Conforme »
la plateforme et la base de l'ERP	Cours 04	les fichiers de `_univers/plateforme/` copiés (D91), la base démarrée et chargée par `charger_erp.sql`, 25 483 commandes
Airflow	Cours 06	l'image construite et les cinq services sains, avec le pool `entrepot`
Spark	Cours 07	un conteneur à usage unique, sans service à démarrer (D76)
Grafana	Cours 12	le compose, le `Dockerfile.airflow` et le dossier `grafana/` recopiés ; `GRAFANA_MOT_DE_PASSE` dans le `.env` ; l'image d'Airflow refaite avec le pilote ODBC de PostgreSQL ; la base de l'ERP rechargée à la date des sources, mars 2026
TimescaleDB	Cours 08	le service `series`, port 5434 ; il ne sert qu'à la seconde séance, et son bloc peut l'attendre, comme celui d'Airflow au Cours 06
Mosquitto, Kafka	Cours 09	le courtier `mqtt` et son `mosquitto.conf` dès l'ouverture, avec les trois clients Python du chemin chaud ; Kafka, un seul nœud en mode KRaft, au début de la seconde séance, avec le mot de passe de la base dans le `.env` du dépôt

Cours 00 — Générer les données sources

Un seul support, `Cours_00_DE_Boreal.md`, de la forme des cours de La Brûlerie : les outils vérifiés, le projet `Projet_Boreal/` créé, la génération de Boréal lancée et son bilan lu, verdict « Conforme », puis les six dossiers de sources listés. Aucune théorie ni aucun exercice : le Cours 01 commence dans le projet ainsi monté.

Cours 01 — Analyser les sources

© 2026 iAlexMG. Tous droits réservés.

Le métier, ce qu'il produit, à qui il le livre. L'architecture de référence (source, ingestion, lac, entrepôt, restitution) et ses deux chemins, le froid et le chaud. La chaîne du parcours et les produits que cette formation y livre. Boréal et ses six sources, vus depuis l'amont. DuckDB, qui interroge un fichier sur place, sans serveur et sans rien charger. Et le piège qui ne plante pas : un type deviné. Lu sans précaution, un extrait de l'ERP garde ses numéros en texte, mais sa date de création `ERDAT` devient un entier (20240531), comme le mandant `MANDT`, alors que `AEDAT`, où traîne la date vide `00000000`, reste du texte : deux colonnes de dates, deux types, et aucune erreur ne l'annonce.

Élément	Détail
Données	le premier regard en SQL : l'historien (275 436 lignes ; qualités 192, 64 et 0 pour 275 080, 35 et 321 lignes, valeurs de départ comprises), les extraits de l'ERP (26 538 en-têtes de commande lus dans les extraits, pour 25 483 dans la base : un extrait n'est pas la base), la télémétrie (158 146 messages) ; le poids de ce qui est arrivé ; un `VBAK` d'extrait lu avec des types devinés, puis en texte
Événements	E6 se voit dans la liste des dossiers (`2023-03_bis`), sans être traité
Le labo livre	le dépôt `01_de` et son `.gitignore` ; les premières requêtes ; le poids des sources, 13,7 Mo d'historien et 70,6 Mo de télémétrie ; le type deviné, constaté sur l'ERP et la télémétrie
Contrôle	aucun : rien ne s'écrit
Variantes	aucune
CIMA+	chemin chaud et chemin froid, posés ; DuckDB

Cours 02 — Ranger les fichiers en bronze

Le lac : ligne contre colonne. CSV, JSON, Avro, Parquet, ORC : ce que chacun coûte et ce qu'il rend. Compression, schéma embarqué, prédicat poussé à la lecture. La règle du bronze, qui répond au type deviné du Cours 01. Le stockage objet, le partitionnement par chemin et le piège des petits fichiers. DuckDB pour écrire et interroger du Parquet sur place.

Élément	Détail
Données	l'historien de 92 jours converti de CSV en Parquet partitionné par jour ; la télémétrie de JSON lines en Parquet imbriqué, et ses 884 fichiers quotidiens face au piège des petits fichiers

© 2026 iAlexMG. Tous droits réservés.

Élément	Détail
Le labo livre	les deux scripts de `01_de/sql/` ; le bronze de l'historien et de la télémétrie dans `plateforme/lac/bronze/`, la mesure de ce que le format change en taille et en vitesse
Règle du bronze	chaque ligne et chaque valeur telles que reçues ; un type déclaré quand la conversion ne perd rien, du texte ailleurs ; compression zstd, un dossier par jour livré
Contrôle qui échoue	les lignes du CSV contre celles du Parquet ; une date `DATS` qui doit rester du texte
Variantes	encadrés Snowflake et Databricks
CIMA+	DuckDB et Parquet

Les Cours 01 et 02 viennent de la coupe de l'ancien Cours 01, le 2026-09-18, à la charnière « regarder, puis écrire » (D99). Le premier finit sur un constat mesuré, le type deviné, sans rien écrire dans le lac ; le second ouvre sur la règle qui en découle, et écrit le bronze.

Cours 03 — Charger la `RAW`

Extraire d'un dépôt de fichiers, la voie la plus répandue : les extraits mensuels de l'ERP et des garanties, l'archive de la télémétrie, les exports de l'usine, l'instantané de la météo. Chargement complet contre chargement incrémental. La propriété qui gouverne tout, l'idempotence : un journal des chargements, et rejouer un extrait ne double rien. La couche `RAW` et ses règles. Et ce qui se dépose tel quel sans devenir une table : la documentation.

Élément	Détail
Données	les extraits de 2022-01 à 2025-12, puis de 2026-01 à 2026-03 (E17) ; la télémétrie archivée ; le référentiel et la GMAO de l'usine ; l'instantané de la météo ; le corpus
Le piège qui ne plante pas	`2023-03_bis` porte un autre nom que `2023-03` : le journal des chargements le laisse passer, et 609 commandes arrivent deux fois. Le cours le montre, le contrat du Cours 11 l'attrape, et la `RAW` le garde marqué de son lot pour que le test d'unicité d'AE ait quelque chose à prouver
Le labo livre	la `RAW` : 22 tables (9 de l'ERP, 5 des garanties, 3 du référentiel et de la GMAO, 3 de la météo, la télémétrie, le catalogue du corpus) ; le corpus déposé dans le bronze ; `LIVRAISON.md` commencé

© 2026 iAlexMG. Tous droits réservés.

Élément	Détail
Contrôle	charger deux fois le même extrait ne change aucun compte
Poste	Windows seul : l'Anaconda Prompt et DuckDB
Variantes	annexe Snowflake, annexe Databricks

Cours 04 — Extraire d'une base ou d'une API

Extraire d'une base relationnelle par ODBC : la base de l'ERP, dans son conteneur PostgreSQL ; le pilote comme contrat, et pourquoi on ne lit pas une base de production à l'heure de pointe. D'une API REST : la vraie API d'ECCC (`api.weather.gc.ca`), sa pagination, sa limitation de débit, la reprise sur erreur, comparée à l'instantané figé. Filigrane sur la date de modification des commandes (`AEDAT`) ; capture de changements par requête, sur les documents de modification `CDHDR` et `CDPOS`, et la capture par journal annoncée (la synthèse Aurore la fait). MQTT, REST et ODBC comparés : qui pousse, qui tire, ce que chacun garantit ; MQTT est traité au Cours 09.

Élément	Détail
Données	la base au 2025-12-31 (25 483 commandes), puis au 2026-03-31 (27 185) ; les 49 documents de modification du palier du 2024-07-01 (E8) ; la vraie API d'ECCC
Le labo livre	la plateforme reçue et la base de l'ERP montée ; ses neuf tables tirées par ODBC, à 0 écart de la `RAW` ; le filigrane de décembre égal à l'extrait ; 92 jours de météo tirés de l'API
Contrôle	l'état reconstruit depuis les extraits égale la base sur les neuf tables
Poste	Ubuntu et Docker pour la base, le pilote ODBC, Internet pour l'API
Variantes	aucune
Avant d'écrire	le correctif de `verifier_sql.py` (D24) ; l'essai de `lancer_depuis` hors de la formation et d'un `.sql` à deux moteurs ; l'essai d'ODBC depuis Windows vers le conteneur de WSL 2 ; les faits des plateformes et de l'API d'ECCC revérifiés
CIMA+	REST, ODBC, et MQTT, REST et ODBC comparés

Les Cours 03 et 04 viennent de la coupe de l'ancien Cours 02, le 2026-09-18, à la charnière « fichiers, puis base et API » (D99) : le lecteur finit le premier sans avoir ouvert Ubuntu.

© 2026 iAlexMG. Tous droits réservés.

Cours 05 — Rendre le pipeline robuste

Ce qui sépare un script d'un pipeline : la configuration hors du code, les secrets hors du dépôt (le mot de passe de la base de l'ERP), une journalisation exploitable, les reprises avec temporisation exponentielle (l'API d'ECCC), les tests d'une transformation, l'empaquetage et l'exécution en conteneur. Le langage Python lui-même reste à la formation Python.

Une règle de conception sert tout le parcours : le pipeline a une seule porte d'entrée en ligne de commande, que le DAG du Cours 06 appelle et que le kit 02 rejoue sans Airflow. Le chemin que rejoue le kit n'importe que DuckDB et la bibliothèque standard.

Élément	Détail
Données	les ingestions des Cours 02 à 04, reprises en paquet Python dans `01_de/`
Le labo livre	la configuration, le `.env` hors du dépôt, les journaux, les reprises, des tests `pytest` (le journal refuse un extrait déjà chargé, la conversion garde le compte des lignes, la reprise s'arrête après le nombre d'essais prévu), une image Docker du pipeline lancée sur `sources/` monté
Variantes	aucune

Cours 06 — Orchestrer et rattraper

Le graphe acyclique dirigé comme façon de penser. Tâches, dépendances, planification, intervalle de données, rattrapage, nouvelle tentative, capteurs, alertes de retard. Ce qui rend un DAG rejouable, et pourquoi un DAG qui n'est pas idempotent est une bombe à retardement. La couture avec `dbt build`.

Élément	Détail
Données	un DAG quotidien (historien, télémétrie, météo) et un DAG mensuel (extraits de l'ERP et des garanties, compteur)
Événements	les mois de 2026 (E17) : régénérer avec `--jusqua 2026-03`, laisser Airflow rattraper trois mois, et constater que l'état égale un chargement d'un bloc
Le labo livre	les deux DAG dans `01_de/`, Airflow 3 en conteneurs ; la tâche qui lancera `dbt build`, posée en attente : AE la remplit
Variantes	encadrés Snowflake et Databricks
Avant d'écrire	fait le 2026-09-16 : la pile tient, l'entrepôt de Windows s'écrit depuis un conteneur, le rattrapage égale le bloc, et les SLA d'Airflow 2 sont devenus les Deadline Alerts. Restent quatre choix de forme à trancher

© 2026 iAlexMG. Tous droits réservés.

Cours 07 — Traiter un gros volume

Quand une machine ne suffit plus, et surtout quand elle suffit encore. Partitions, évaluation paresseuse, brassage, l'API DataFrame. Lire le plan d'exécution. Les pièges classiques : le brassage inutile, la partition déséquilibrée, le `collect` qui ramène tout au pilote.

Élément	Détail
Données	l'historien complet (731 jours, 2 133 145 lignes, 106 Mo, `--historien complet`) et la télémétrie : le même traitement en DuckDB et en Spark (conversion en Parquet partitionné, agrégats quotidiens par étiquette, messages dédoublonnés par jour), mesuré
Ce que le cours conclut	DuckDB suffit à Boréal. Le cours dit d'où viendrait le besoin (des dizaines d'usines, des années de points bruts), sans le simuler
Partition déséquilibrée	les étiquettes qui bougent sans cesse contre les états de marche, qui ne gardent presque aucun point
Variantes	annexe Databricks, encadré Snowpark
Avant d'écrire	fait le 2026-09-16 : Spark 4.2.0 en conteneur rend les mêmes chiffres que DuckDB, 27 à 120 fois plus lentement, et les limites de Free Edition sont revérifiées. Restent deux choix de forme à trancher
CIMA+	Databricks, repris

Cours 08 — Lire des séries temporelles

D'où vient la donnée industrielle : le capteur, l'automate, le SCADA qui supervise et commande, OPC UA comme langue commune des équipements. L'historien, qui archive les points à haute fréquence : compression par bande morte, qualité du point, étiquettes et hiérarchie des actifs. Les bases de séries temporelles ouvertes : ce qu'elles font mieux qu'un entrepôt, et moins bien. Les pièges du terrain : trous, horloges, heure locale, rééchantillonnage, unités. Et la frontière qui gouverne tout : on ne branche jamais un pipeline sur le réseau de contrôle ; la séparation des technologies opérationnelles et des technologies de l'information existe pour la sécurité.

Élément	Détail
Données	l'usine de Val-des-Aulnes : 6 zones, 17 équipements, 58 étiquettes ; l'historien de 182 jours que le Cours 07 laisse (550 087 lignes), le référentiel, la GMAO (825 bons), le compteur (148 891 relevés)

© 2026 iAlexMG. Tous droits réservés.

Élément	Détail
Les pièges, un à un	les valeurs de départ (10 556 lignes, 58 par jour) ; l'archivage forcé de minuit (10 525 points, et non 10 556 : quatre jours en perdent 31) ; le retour à l'heure normale du 2025-11-02 (E1, 49 paires en double) et le passage à l'heure avancée du 2026-03-08, qui laisse une heure vide ; l'humidité de la cabine de peinture figée 49,8 heures à qualité bonne (E2), et son faux jumeau, la ligne des motomarines à l'arrêt 790 heures ; 529 points de qualité mauvaise, sans valeur, et 64 de qualité douteuse (E3) ; le compteur horodaté en fin d'intervalle avec son décalage à UTC, et 247 relevés estimés
L'argent	les points dédoublonnés, l'horodatage résolu en UTC, la qualité portée ; le rééchantillonnage horaire (moyenne pondérée par le temps pour une mesure, dernier état pour une marche, différence pour un compteur) ; l'énergie horaire du compteur. Le grain exact se fixe au cours
L'incident	`UTL.CMP2.VIBRATION` monte dès le 2025-10-15 ; le compresseur 2 déclenche le 2025-10-28 à 10 h 44 (bon `BT-2025-00742`)
Le labo livre	`plateforme/lac/argent/usine/`, le schéma `argent`, TimescaleDB chargée (hypertable, agrégat continu horaire), comparée à DuckDB
Variantes	aucune
Avant d'écrire	l'essai de TimescaleDB, fait le 2026-09-20 ; celui de Grafana, fait avec le Cours 12
CIMA+	SCADA et historien, base de séries temporelles

La charnière, posée le 2026-09-18 avant l'écriture (D93, D99) : d'où viennent les points, puis ce qu'on en sert. La formation compte douze cours, et celui-ci reste un seul cours : la charnière y pose une coupure nommée entre deux séances d'environ deux heures et demie, comme le Labo 06 après son étape 5. Chaque séance se tient seule. La première finit sur des pièges comptés, la seconde sur une série horaire prouvée.

	Première séance : le terrain et ses pièges	Seconde séance : l'argent et la base de séries temporelles
Théorie	le capteur, l'automate, le SCADA et OPC UA ; l'historien, la bande morte, la qualité du point, les étiquettes et la hiérarchie des actifs ; la frontière entre technologies opérationnelles et technologies	le nettoyage : points dédoublonnés, heure résolue en UTC, qualité portée ; le rééchantillonnage selon la nature du point ; les bases de séries

© 2026 iAlexMG. Tous droits réservés.

	Première séance : le terrain et ses pièges	Seconde séance : l'argent et la base de séries temporelles
	de l'information ; les pièges du terrain, trous, horloges, heure locale et unités	temporelles, ce qu'elles font mieux qu'un entrepôt et moins bien, et la licence de l'agrégat continu
Labo	le référentiel, la GMAO et le compteur ; les six pièges de la ligne « Les pièges, un à un », mesurés sur le bronze de l'historien et sur le compteur ; l'incident, lu dans les points et dans le bon de travail	l'argent de l'usine dans `plateforme/lac/argent/usine/` et dans le schéma `argent` ; l'énergie horaire du compteur ; TimescaleDB chargée, avec son hypertable et son agrégat continu
Fin de la séance	chaque piège compté, au chiffre de la fiche	l'agrégat continu égale le rééchantillonnage de DuckDB
Service	aucun	TimescaleDB

Cours 09 — Ingérer un flux d'événements

Le contraste avec le lot : ce que le temps réel apporte, ce qu'il coûte. MQTT, le protocole léger du terrain : un courtier, la publication et l'abonnement, des sujets hiérarchiques, trois niveaux de qualité de service, le message retenu. Kafka, quand le flux doit être conservé et relu : sujets, partitions, clés, décalages, groupes de consommateurs. Le pont entre les deux. Sémantique de livraison, fenêtrage, données en retard. L'architecture Lambda, qui garde les deux chemins séparés, et Kappa, qui ne garde que le flux. Et la question honnête : la plupart des besoins dits « temps réel » sont des besoins horaires.

Élément	Détail
Données	les rejoueurs de `_univers/rejoueurs/rejouer.py`, écrits à la séance de ce cours, sur la génération `--jusqua 2026-03` : l'usine du 28 octobre 2025 (4 048 points en 24 s) et du 13 au 31 octobre (62 726 en 456 s), la télémétrie du 10 au 16 janvier 2026 (3 449 messages en 168 s), à 3 600 fois
Événements	sur la semaine rejouée, 70 doublons de la qualité de service 1 et 246 messages reçus plus d'une heure après la fin de leur sortie (E12), jusqu'à 4 jours ; la dérive du compresseur 2 (E5), dont l'alerte se lève le 2025-10-22 à 8 h 15
Le labo livre	le paquet `flux` du dépôt : l'abonné, le pont MQTT vers Kafka qui n'acquitte qu'après Kafka, le consommateur qui écrit le schéma `temps_reel` de TimescaleDB, les fenêtres de 15 minutes et l'alerte à bande morte ;

© 2026 iAlexMG. Tous droits réservés.

Élément	Détail
	`sql/temps_reel.sql` ; trois tests de plus. L'alerte se lève 6 j 2 h 29 min avant le déclenchement, l'arrêt 1 j 4 h 29 min
Ce que le cours montre aussi	l'archive quotidienne de la télémétrie est ce qu'écrit un abonné : réécoutée, elle revient ligne pour ligne, 0 écart ; un groupe Kafka neuf relit octobre en moins d'une seconde
Variantes	encadrés Snowflake (Snowpipe Streaming, streams et tâches) et Databricks (Structured Streaming, `AvailableNow` seul sur serverless), d'après leur documentation consultée le 2026-09-23
Avant d'écrire	fait le 2026-09-23 : les rejoueurs prouvés, Mosquitto 2.1.2 et Kafka 4.3.1 en mode KRaft mesurés, le relais de WSL 2 qui retient les requêtes de Kafka de plus de 200 octets
CIMA+	MQTT ; chemin chaud et chemin froid, Lambda et Kappa

La charnière, posée le 2026-09-18 avant l'écriture (D93, D99) : le terrain qui publie, puis le flux qu'on garde et qu'on relit. Comme au Cours 08, un seul cours et une coupure nommée entre deux séances. La première finit quand un abonné reçoit ce que les rejoueurs publient, doublons et retards compris ; la seconde garde le flux dans Kafka, le relit par fenêtres et en tire l'alerte.

	Première séance : MQTT, le terrain qui publie	Seconde séance : Kafka, le flux qu'on garde
Théorie	ce que le temps réel apporte et ce qu'il coûte, et la question honnête des besoins horaires ; MQTT : le courtier, la publication et l'abonnement, les sujets hiérarchiques, les trois niveaux de qualité de service, le message retenu ; la sémantique de livraison	Kafka : sujets, partitions, clés, décalages, groupes de consommateurs ; le pont entre les deux ; le fenêtrage et les données en retard ; Lambda et Kappa
Labo	les rejoueurs de l'usine et de la télémétrie ; l'abonnement ; les doublons de la qualité de service 1 et les messages en retard, vus à la réception ; l'archive quotidienne reconnue comme ce qu'écrit un abonné	le pont MQTT vers Kafka ; le consommateur qui écrit dans TimescaleDB ; la fenêtre de 15 minutes sur la vibration du compresseur 2, et son alerte
Fin de la séance	les messages publiés égalent l'archive	l'alerte datée pendant la dérive, avec son avance sur le déclenchement

© 2026 iAlexMG. Tous droits réservés.

	Première séance : MQTT, le terrain qui publie	Seconde séance : Kafka, le flux qu'on garde
Service	Mosquitto	Kafka en mode KRaft

Cours 10 — Passer à un format de table ouvert

Iceberg et Delta : des transactions ACID sur des fichiers. Évolution de schéma, voyage dans le temps, compactage. L'architecture médaillon : le bronze garde la donnée telle que reçue, l'argent la nettoie et la rend conforme, l'or la prépare pour la consommation. La correspondance avec `RAW`, `STAGING` et `MARTS`, l'endroit où elle cloche, et la réponse du parcours : DE tient l'argent des séries de terrain, et les sources relationnelles passent de la `RAW` au `STAGING` d'AE (D32). La convergence du lac et de l'entrepôt, et les lakehouses clés en main.

Élément	Détail
Données	la télémétrie du bronze, sur la génération `--jusqua 2026-03` : 933 fichiers quotidiens et 204 485 messages, en table Iceberg. Le micrologiciel 2.0 (E11, dès le 2025-05-01, 115 807 messages, dont 69 468 avant 2026) ajoute `heures_moteur` et renomme `temp_moteur_max` en `temperature_moteur_max`, et 9 835 messages portent encore l'ancien nom après le 1er mai : c'est le cas d'évolution de schéma. 2025 en 28 instantanés mensuels, les trois mois de 2026 en 90 instantanés quotidiens, le voyage dans le temps vers l'état au 2025-12-31 ; les séries de l'argent en Iceberg
Le labo livre	le paquet `tables` du dépôt : le catalogue SQL, le chargement de la télémétrie, l'entretien (historique, étiquette, compactage, expiration, nettoyage), l'argent en Iceberg et la même télémétrie en Delta ; `sql/lac_iceberg.sql`, qui ouvre les tables dans DuckDB par le catalogue lu comme une base SQLite
Ce que le cours montre aussi	le renommage par identifiant, vu dans le fichier Parquet (colonne 9) et dans les schémas successifs du fichier de métadonnées ; un fichier lu sur 28 par la partition cachée ; le voyage dans le temps par la date qui rend 0 message, parce qu'il suit l'heure des validations ; les métadonnées qui finissent plus lourdes que les données ; la partition d'avril des séries de l'usine, le mois se comptant en UTC ; `deltalake` 1.6 qui refuse la correspondance des colonnes
Variantes	annexe Snowflake (table Iceberg dont Snowflake est le catalogue, volume `SNOWFLAKE_MANAGED`) et annexe Databricks (correspondance des colonnes, historique, clone profond), encadrés Snowflake, Databricks et Fabric, d'après leur documentation consultée le 2026-09-23 ; rien n'y a été lancé

© 2026 iAlexMG. Tous droits réservés.

Élément	Détail
Avant d’écrire	fait le 2026-09-23 : PyIceberg 0.12 et `deltalake` 1.6.5 essayés sous Windows, avec un catalogue SQL local lu par DuckDB 1.5.5
CIMA+	médaillon, formats de table ouverts, Fabric

Cours 11 — Contrôler la qualité

Le contrat de données entre producteur et consommateur. Valider à l’entrée plutôt que déboguer en aval. Fraîcheur, volume, distribution, unicité, et pour un capteur, la plage physiquement possible. Bloquer ou avertir, et pourquoi ce n’est pas le même choix pour chaque contrat. Lignage, alerte utile contre alerte ignorée, conduite d’un incident de données. La parenté avec les tests dbt, dite explicitement.

Élément	Détail
Données	un contrat par produit livré, qui porte ses attendus (D64)
Ce que les contrats attrapent	l’extrait `2023-03_bis` identique à `2023-03` (E6) ; la commande `0010015405` d’un concessionnaire absent de l’extrait `2024-05` (E7) ; le champ renommé du micrologiciel 2.0 (E11) ; le capteur figé (E2) ; les valeurs hors de la plage de `etiquettes.csv` et les coupures (E3) ; la pression instable du réseau d’air pendant la dérive du compresseur 2, par un contrôle de distribution sur les heures de production (E5)
Le labo livre	les contrats, leur exécution dans le DAG, le compte des alertes vraies et fausses, le post-mortem d’un incident de données
Variantes	encadrés Snowflake et Databricks
Avant d’écrire	le contrôle de la pression bâti sur la vraie sortie, et ses fausses alertes comptées hors de la dérive

Cours 12 — Rebâtir le pipeline de bout en bout

Détail en section 7.

7. Le capstone

Il boucle le projet Boréal de la formation (D14) : les deux chemins, de bout en bout, sur les sources générées jusqu’au 2026-03-31.

© 2026 iAlexMG. Tous droits réservés.

Chemin	Ce qui tourne	Ce qui le prouve
Froid	Airflow orchestre l'ERP par ses extraits et l'extraction ODBC qui retrouve la même base, l'historien, la GMAO, le compteur, les garanties, la météo par l'API, la télémétrie archivée et le corpus ; le bronze, l'argent et la `RAW` ; les contrats qui bloquent	le projet se reconstruit de zéro ; l'état final égale `LIVRAISON.md` ; chaque contrat rend son attendu
Chaud	les rejoueurs de l'usine et de la télémétrie, Mosquitto, le pont, Kafka, TimescaleDB, Grafana et l'alerte de vibration	l'alerte se lève pendant la dérive, à la date calculée au Cours 09

Ce que le capstone livre : le dépôt `01_de` complet, la pièce de portfolio ; `LIVRAISON.md` complet ; le tableau de bord Grafana de l'usine (état du réseau d'air, vibration et température du compresseur 2, alerte), que BI reprend pour en concevoir la lecture ; le `Solution/` que rejouent les kits 02 et 03, ce dernier gagnant au lot 7 TimescaleDB chargée et le rejoueur. L'incident vu depuis l'amont tient en trois faits : la date de l'alerte, le contrat qui a vu la pression instable, et une `RAW` où AE retrouvera la plage des motoneiges touchées.

Écrit le 2026-09-23. Le chemin froid se refait par un DAG sans horaire, `boreal_reconstruction`, en sept tâches et 3 min 12 s, après une étape `argent` ajoutée au pipeline ; trois preuves le jugent : le kit 02 et son option `--verifier`, la comparaison des lignes dans les deux sens contre l'entrepôt mis de côté, 26 tables et 0 écart, et les contrats, 23 règles, plus la preuve ODBC du Cours 04. Le chemin chaud se rejoue de zéro, les tables du temps réel vidées : 62 726 points, l'alerte le 2025-10-22 à 8 h 15. Le tableau de bord de l'usine vit dans `01_de/grafana/`, et le dépôt reçoit son `README.md` et l'étiquette `v1.0`. Ce que le capstone ne refait pas, il le dit : les versions remplacées que le journal des chargements raconte, que les sources ne portent plus, et les tables Iceberg et Delta, qui se refont par leurs commandes.

8. La synthèse, et le mini-cas

D96 n'en garde qu'une par formation, la plus avancée : Aurore. Elle suit le capstone (D14), repart des données brutes générées (D15) et suit le modèle d'AE (D16) : une mise en situation, un document unique qui refait l'arc de la formation d'un bloc, un dossier `Solution/` rejouable, sans notes ni examens.

La Brûlerie change de rôle : elle devient le mini-cas figé du premier pas, et non plus une synthèse (D96, D110). Ses données ne se génèrent plus chez le lecteur : la sortie de `--extrait mois` de juin 2025, avec `graines/` et ses empreintes, est versionnée, et le générateur quitte son chemin.

8.1 La Brûlerie, le mini-cas figé

`01_Brulerie_facile/` : `Mise_en_situation_La_Brulerie_01_DE.md` (D126), `Synthese_brulerie.md`, `Solution/`. Données : le mini-cas figé, `_univers/brulerie/mini_cas/sources/`, un mois de caisse versionné, 100 fichiers et 0,9 Mo (A5.10). Le lecteur ne génère rien, et l'arc se refait sur ce mois-là.

© 2026 iAlexMG. Tous droits réservés.

Écrit le 2026-09-23, en Python et DuckDB seulement, sans Docker : le dépôt `01_de` du mini-cas porte un paquet `brulerie` bâti comme `pipeline`, une porte d'entrée et un module par étape, deux scripts SQL et un test. Le périmètre a deux conséquences sur le plan d'origine : le torréfacteur n'est pas rejoué en MQTT, il se lit en fichiers, et l'orchestration quotidienne devient une commande que le Planificateur de tâches de Windows ou cron lancent chaque matin.

Arc refait	**Sur La Brûlerie**
Premier regard	les types que DuckDB devine, et la somme de l'encaissé en `DOUBLE`, qui change de dernier chiffre d'une exécution à l'autre
La `RAW`	les neuf livraisons tout en texte, avec leur journal et leurs empreintes, le lot lu dans le nom du fichier ; les deux exports du 2025-06-12 gardés, chacun sous son lot
Le bronze	la caisse en Parquet, une partition par jour ; l'export repris du 2025-06-12 (L5) : le dernier export d'un jour fait foi, et la partition se vide avant d'être réécrite
La fenêtre	une commande qui reçoit sa tranche, et `--par-jour`, qui rattrape les jours manqués comme un ordonnanceur ; le mois jour par jour égale le mois d'un bloc
Terrain	le torréfacteur tiré de ses courbes, une ligne par lot au schéma `argent`, et un témoin qui recoupe son journal ; un lot n'est dû que quand son jour est livré
Contrat	neuf règles de la caisse, toutes bloquantes, jugées sur tout le bronze ; un fichier égaré dans la partition du 12 juin fait sortir 58 écarts
Ce qu'on n'installe pas	ni Airflow, ni Spark, ni MQTT et Kafka, ni Iceberg : un petit commerce n'en a pas besoin, et le mini-cas le dit

La preuve : le `Solution/` rejoué sur deux projets neufs, le mois jour par jour en 30 exécutions et 6 s, le mois d'un bloc en 2,3 s, rend deux projets identiques, 11 tables et 10 327 lignes dans l'entrepôt, 60 fichiers Parquet et 4 059 lignes dans le bronze, 0 écart. Les totaux du bronze égalent ceux de la caisse, section 2.1 de `_univers/brulerie/CHIFFRES_ATTENDUS.md` : 1 527 tickets, 16 718,00 $ avant taxes, 18 666,38 $ encaissés. Les 47 lots passent le témoin sans écart. Aux taux de `_commun/outils/mesurer_duree.py`, le mini-cas demande 67 minutes de lecture et 78 de frappe : une séance.

8.2 Aurore, avancée

`03_Aurore_avancee/` : `Mise_en_situation_Aurore_01_DE.md` (D126), `Synthese_aurore_DE.md`, `Solution/`. Données : `_univers/aurore/generer_aurore.py` (40 Mo, 305 Mo décompressés), puis `--jusqua 2026-03`.

Écrite le 2026-09-23, en Python et DuckDB seulement, sans Docker : le dépôt `01_de` de la synthèse porte un paquet `aurore` bâti comme `brulerie`, une porte d'entrée et un module par étape, deux scripts SQL et un test. Le périmètre change deux choses au plan d'origine : la copie de la base n'est pas chargée dans

© 2026 iAlexMG. Tous droits réservés.

PostgreSQL, elle sert de schéma et de témoin, et les événements web ne passent pas par Kafka. Le journal et le web arrivent déjà en fichiers de messages, et la logique d'un consommateur, doublons, ordre et retards, reste la même.

Arc refait	**Sur Aurore**
La `RAW`	le journal de capture, les événements web et les taux de change réels de la Banque du Canada, chaque message gardé entier dans une colonne JSON, avec son journal des chargements
Capture de changements par journal	la base rebâtie de son journal, schéma `boutique` : le dernier changement de chaque ligne par `source.lsn`, les suppressions, l'instantané initial ; le type de chaque colonne tiré du DDL livré avec la copie, faute de schéma dans le message, les décimaux en texte et les dates en jours depuis 1970
Idempotence	les 127 changements republiés au redémarrage du connecteur, le 2024-10-08 (A5), sans effet sur l'état puisqu'ils portent le même `lsn`
Évolution de schéma	la colonne ajoutée à la table des commandes le 2025-01-20 (A6) : vide avant, et gardée par une règle du contrat qui compare les colonnes du journal au DDL
Continu	les 408 074 événements web lus jour par jour : 1 669 doublons écartés par leur identifiant, le retard mesuré, la fenêtre quotidienne et son filigrane ; la version 2 du format (A7, 2025-06-02) attrapée par un contrat ; le robot du 2024-09-09 au 2024-09-20 (A8) signalé sans être retiré, parce que l'écarter est une décision de BI et de DS
Preuve	la base rebâtie égale la copie de la base, table par table, valeur par valeur ; `preuves/verifier_journal.py` le redit par un autre chemin
Allongement	la période portée à mars 2026 : le dernier fichier du journal, qui change, repris par son empreinte ; le projet allongé égal au même chargé d'un bloc

La preuve : sur une génération neuve, les 18 tables de la base rebâtie égalent la copie, 115 667 lignes ; la fenêtre qui s'arrête au 31 décembre laisse 7 tables en écart, 28 lignes absentes et une périmée, parce que la copie est prise au soir du 31, heure de Montréal, et que le dernier fichier du journal est celui du 1er janvier UTC. Les sept règles bloquantes du contrat passent ; un contrat qui ne connaît que la version 1 du format web bloque le 2025-06-02 sur 430 événements, et un DDL d'avant A6 sur la colonne inconnue. Allongé jusqu'à mars 2026, le projet égale le même chargé d'un bloc : 22 tables, 1 191 657 lignes, 0 écart. Aux taux de `_commun/outils/mesurer_duree.py`, la synthèse demande 84 minutes de lecture et 83 de frappe : deux séances.

© 2026 iAlexMG. Tous droits réservés.

Le rejoueur de `_univers/rejoueurs/` couvre deux flux de Boréal, l'usine et la télémétrie, par un seul script, qui lit un flux généré et le publie sur MQTT. Ni le torréfacteur de La Brûlerie, ni le web d'Aurore n'en ont eu besoin : le mini-cas et la synthèse les lisent en fichiers. Il dépend de paquets déclarés, comme le permet la règle des générateurs.

9. Le kit 02

La sortie de DE devient l'entrée d'AE (D10, D11, D35) : le kit reconstruit ce que DE aurait livré, en rejouant le `Solution/` de DE.

Élément	**Détail**
Commande	`python _univers/kits/demarrer.py --formation 02 --projet Projet_Boreal`
Ce qu'il fait	génère les sources si elles manquent, ou vérifie leurs empreintes ; rejoue le chemin froid du `Solution/` par sa ligne de commande, sans Airflow, Kafka, Spark ni Docker ; écrit `raw` et `argent` dans l'entrepôt, le lac bronze et argent, une copie Parquet de chaque table livrée ; affiche l'état, le compare aux attendus et passe les contrôles de `LIVRAISON.md`
Options	`--jusqua 2026-03` ; `--historien complet`, pour le kit 04 ; `--verifier`, qui passe les contrôles de DE sur le projet d'un apprenant (M3) ; `--catalogue`, le catalogue du script Databricks
Dépendances	`duckdb` seulement
Durée	2 minutes visées ; la durée mesurée, génération comprise, est dans `_univers/kits/README.md`
Plateformes	un script de chargement généré pour Snowflake, un pour Databricks, sans identifiant de compte ; vous les exécutez
Preuve	deux exécutions sur une copie jetable du projet, mêmes comptes et mêmes sommes de contrôle, l'heure de chargement exclue ; le chargement sur Databricks Free Edition, fait par vous

© 2026 iAlexMG. Tous droits réservés.